

AI-Enhanced Scholarship Management System

Project Documentation

[Github Repository Link](#)

[Deployment Link](#)

Prepared using the provided reference document and the actual project source code.

Project Title	AI-Enhanced Scholarship Management System
Domain	Scholarship Management / AI Decision Support
Primary Stack	Flask, PostgreSQL, scikit-learn, HTML/CSS/JavaScript
ML Model	Logistic Regression
AI Support	LLM-based explanation and reporting
Deployment Support	Docker, Render, Railway, Vercel

Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Literature Survey and Scope of the Project
5. Proposed System
6. System Design and Implementation
7. Technology Stack
8. Database Design
9. Frontend Design
10. Backend and API Design
11. Machine Learning Module
12. Generative AI Module
13. Algorithm Used
14. Implementation Details
15. Results and Screenshots
16. Conclusion
17. Future Enhancements
18. References

1. Abstract

The AI-Enhanced Scholarship Management System is a full-stack web application that digitizes and improves the scholarship application lifecycle. The platform allows students to register, log in, submit scholarship applications, attach supporting documents, and track the status of their submissions. For administrators, the system provides a review dashboard, eligibility prediction, approval and rejection workflow, analytics, exported reports, and notification support.

The project combines Flask for the backend, PostgreSQL for persistence, HTML/CSS/JavaScript for the frontend, and a machine learning module built with scikit-learn. A Logistic Regression model estimates whether a student is eligible for scholarship consideration using attributes such as GPA, attendance, family income, category, previous scholarship history, and extracurricular involvement. In addition, an AI text generation layer produces human-readable explanations and review reports so that scholarship decisions can be presented in a clear and transparent manner.

The system also includes password reset support, automatic initialization of database tables, secure password hashing, file upload handling for application documents, and downloadable admin exports. Together, these features reduce manual work, improve consistency, and support a more transparent decision-making process for both students and scholarship officers.

2. Introduction

Scholarship programs play an important role in helping deserving students pursue higher education. However, many scholarship offices still rely on spreadsheets, manual forms, or loosely connected tools to manage applications. This approach often creates delays in application review, inconsistent decisions, and limited visibility for applicants.

The AI-Enhanced Scholarship Management System addresses these issues by centralizing the entire workflow in one application. Students can register and apply through a web interface, while administrators can review submissions from a separate dashboard. The system automatically predicts eligibility, stores application data in PostgreSQL, generates AI-based explanations, and tracks approval status.

The main objective of the project is not only to automate the process but also to make it understandable. A common limitation in many automated decision systems is that the output is difficult to explain. This project explicitly includes probability scores and AI-written summaries so that scholarship officers can review and justify decisions with greater confidence.

3. Problem Statement

Scholarship selection is often a time-sensitive and high-responsibility task. In manual systems, staff members need to read each application, compare documents, evaluate eligibility conditions, and prepare approval notes individually. This process becomes slower and more error-prone as the number of applicants increases.

The key problems addressed by this project are:

- No centralized digital workflow for student registration, application submission, and review.
- Manual evaluation of applicant profiles, which increases workload and may lead to inconsistent decisions.
- Limited transparency for students regarding why an application is considered eligible or ineligible.
- Difficulty in generating reports, summaries, and decisions across many applications.
- Lack of AI assistance that can help explain or summarize scholarship decisions in natural language.

To solve these challenges, the project introduces an intelligent scholarship management platform that combines role-based access, automated prediction, document handling, analytics, and AI-generated explanations.

4. Literature Survey and Scope of the Project

Traditional scholarship processing systems typically fall into one of three categories: manual paper-based workflows, spreadsheet-driven record keeping, or simple database-backed portals. Manual systems are easy to start with, but they are slow, hard to audit, and difficult to scale. Spreadsheet-based systems improve storage but still require a large amount of human intervention.

Many scholarship portals are also limited to form submission and approval tracking. They do not usually provide intelligent eligibility prediction, explainable reasoning, or automatic reporting. As a result, administrators still spend significant time reading each application and preparing decisions.

The scope of the proposed system includes:

- Student registration, login, and password reset support.
- Scholarship application submission with academic and financial fields.
- Automatic eligibility prediction using a trained Logistic Regression model.
- AI-generated explanations for each application.
- Admin review workflow with approve and reject actions.
- Dashboard statistics, summary metrics, and report generation.
- Document upload support and storage of application attachments.

The future scope of the project includes OCR-based document verification, recommendation ranking across multiple scholarship schemes, mobile application support, and more advanced explainable AI methods.

5. Proposed System

The proposed system is designed as a role-based scholarship management platform with two primary user groups: students and administrators. Students use the portal to create accounts, apply for scholarships, and view their submission history. Administrators use a separate dashboard to monitor applications, view eligibility predictions, inspect AI-generated reports, and approve or reject applications.

The system performs an automated analysis whenever a new application is submitted. The backend collects the application fields, sends them to the machine learning service, receives an eligibility prediction and probability score, and then calls the AI text generation layer to produce a short explanation. The application is then stored in the database with a pending review status.

The design emphasizes three goals: automation, transparency, and traceability. Automation reduces manual effort, transparency helps explain decisions to both users and officers, and traceability ensures that the history of each application is stored for later review.

6. System Design and Implementation

6.1 System Workflow

Stage	Description
Student Login / Registration	The user creates an account or signs in using email and password.

Application Entry	The student completes the scholarship form with academic and financial information.
Flask Backend	The backend validates input and stores the request.
ML Prediction	The Logistic Regression model predicts eligibility and probability.
AI Explanation	The text generation layer creates a readable explanation for the result.
Database Storage	The application is stored in PostgreSQL with status and timestamps.
Admin Review	Administrators review applications, add notes, and approve or reject them.
Notifications and Reports	The system can email status updates and generate AI summary reports.

6.2 Architecture Overview

The application follows a layered architecture. The frontend is responsible for user interaction, the backend handles business logic and API endpoints, the database persists all records, the machine learning service performs eligibility prediction, and the AI service generates explanations and reports.

- Frontend: HTML, CSS, JavaScript pages for index, login, register, student dashboard, and admin dashboard.
- Backend: Flask application with blueprints for authentication, applications, dashboards, review, and reporting.
- Database: PostgreSQL tables for users, applications, files, and notifications.
- ML Layer: scikit-learn pipeline trained on scholarship data and saved as a joblib artifact.
- AI Layer: environment-configurable LLM integration with fallback explanation logic.

6.3 Data Flow

- User registers or logs in.
- Student submits scholarship application.
- Backend validates the payload.
- Machine learning service predicts eligibility.
- AI service generates explanation and report text.
- Application is inserted into PostgreSQL.
- Admin dashboard reads the submitted applications and summary metrics.
- Admin action updates the application status and stores reviewer notes.

7. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Builds the student and admin user interfaces.
Backend	Python, Flask, Flask-CORS	Serves REST APIs and application logic.
Database	PostgreSQL, psycopg2	Stores users, applications, files, and notifications.
Machine Learning	scikit-learn, pandas, numpy, joblib	Trains and serves the scholarship eligibility model.

AI Integration	LLM API / OpenAI-compatible chat endpoint	Generates explanations and review reports.
Security	Werkzeug password hashing	Stores user passwords securely.
Deployment	Docker, Gunicorn, Render, Railway, Vercel	Supports local and cloud deployment.

8. Database Design

The application uses PostgreSQL as its primary database. Tables are created automatically during startup through the `init_db` routine in the backend. This makes the project easier to run locally and also reduces setup errors during deployment.

The main tables are listed below:

Table	Purpose	Important Fields
users	Stores student and admin accounts.	id, name, email, password_hash, role, reset_code, reset_expires_at, created_at
applications	Stores scholarship applications and prediction data.	user_id, full_name, gpa, attendance, family_income, category, eligibility_prediction, eligibility_probability, status, reviewer_note
application_files	Stores uploaded document metadata.	application_id, original_name, stored_name, file_path, file_type, created_at
notifications	Stores notices related to application review.	user_id, application_id, title, message, is_read, created_at

The database design supports authentication, application history, file uploads, admin review, password reset tokens, and notification records. Foreign keys are used to maintain relationships between users, applications, and uploaded files.

9. Frontend Design

The frontend is built with simple static pages and a shared script file. The landing page introduces the system and provides navigation to login and registration. The login page supports student and administrator access, while the registration page creates new student accounts.

After login, students are redirected to the student dashboard and administrators to the admin dashboard. The dashboards display live data fetched from the backend API. The UI uses modern glass-style cards, responsive grids, and clear status badges for approved, rejected, and pending applications.

The key frontend pages are:

- `index.html` - landing page with product overview and entry links.
- `login.html` - authentication page with password reset forms.
- `register.html` - student registration page.
- `dashboard.html` - student dashboard for application submission and history.
- `admin.html` - admin dashboard with review queue and AI insights.

10. Backend and API Design

The backend is implemented with Flask and organized using a blueprint-based API module. The app serves both the API and the frontend pages. CORS is enabled so the frontend can communicate cleanly with the backend endpoints.

Important backend modules in the repository:

- Backend/app.py - creates the Flask app, registers routes, and serves frontend pages.
- Backend/routes/api.py - contains authentication, application, review, summary, and report endpoints.
- Backend/database.py - manages PostgreSQL connections and table creation.
- Backend/ml_service.py - loads the trained model and returns predictions.
- Backend/ai_service.py - generates explanations and report text.
- Backend/email_service.py - handles password reset emails and status notifications.

Important API endpoints exposed by the project:

Method	Endpoint	Purpose
GET	/api/health	Returns server and model status.
POST	/api/auth/register	Creates a new user account.
POST	/api/auth/login	Authenticates a user.
POST	/api/auth/request-password-reset	Sends a password reset code.
POST	/api/auth/reset-password	Updates the password using reset code.
POST	/api/applications	Submits a scholarship application.
POST	/api/applications/<id>/files	Uploads application documents.
PATCH	/api/applications/<id>/review	Approves or rejects an application.
GET	/api/applications	Lists student or admin applications.
GET	/api/dashboard/summary	Returns dashboard metrics.
GET	/api/reports/insights	Returns AI-generated insight summaries.
GET	/api/admin/export	Exports application records as CSV or Excel.

11. Machine Learning Module

The machine learning module predicts whether a student is eligible for scholarship consideration. The code in ML/train_model.py builds a synthetic training dataset, preprocesses numeric and categorical fields, and trains a Logistic Regression classifier. The resulting model is saved as a joblib artifact and loaded by the backend at runtime.

The model uses the following features:

- GPA
- Attendance percentage
- Family income
- Previous scholarship history
- Extracurricular involvement
- Category (General, EWS, OBC, SC, ST)

The project stores evaluation metrics in ML/artifacts/scholarship_metrics.json. According to the generated artifact, the model achieved approximately 87.92% accuracy, 98.66% precision, and 84.48% recall on the synthetic dataset.

The pipeline uses imputation for missing values, scaling for numeric features, one-hot encoding for category, and class-balanced logistic regression to improve the handling of imbalanced labels.

11.1 Prediction Flow

- Application data is converted into a data frame.
- Preprocessing transforms the input features.
- The logistic regression model calculates the probability of eligibility.
- The backend converts the probability into an Eligible / Not Eligible label.
- The result is stored and shown on the dashboard.

12. Generative AI Module

The AI module adds explainability to the scholarship workflow. When a student submits an application, the backend asks the AI layer to generate a short explanation of the prediction. When an administrator reviews an application, the backend generates a more detailed report.

The project is designed to work with an OpenAI-compatible chat completion endpoint through environment variables such as LLM_API_KEY, LLM_API_URL, and LLM_MODEL. If these values are not configured, the system falls back to deterministic explanation text so that the full workflow still works offline.

This approach is useful because it preserves functionality during development while also enabling richer language generation in production. The generated text can help scholarship officers understand why an application was flagged as eligible or ineligible.

- Explanation generation for each submitted application.
- Review report generation for admin decision-making.
- Fallback text when no live model key is available.
- Readable summaries that combine prediction and application context.

13. Algorithm Used

13.1 Logistic Regression Classifier

Logistic Regression is used as the core classifier for scholarship eligibility prediction. It is appropriate for this project because the target is binary in nature: eligible or not eligible. It is also fast, easy to interpret, and works well with a mix of numeric and encoded categorical features.

Working principle:

- Input features are standardized and encoded.
- The model computes a weighted linear score.
- The score is passed through a sigmoid function.
- The output probability indicates the chance of eligibility.
- A threshold of 0.5 is used to classify the application.

The model is trained on synthetic scholarship data generated inside the repository. Although synthetic data is used for training and demonstration, the structure of the pipeline allows the same workflow to be retrained later on real institutional data.

14. Implementation Details

14.1 Frontend Implementation

The frontend is designed to be lightweight and responsive. JavaScript handles form submission, API calls, session storage, dashboard updates, and logout flow. A single script file coordinates login, registration, student dashboard rendering, and admin dashboard rendering.

- Session data is stored in localStorage.
- API requests are made using fetch with JSON payloads.
- Student applications refresh automatically after submission.
- Admin review actions are submitted using PATCH requests.

14.2 Backend Implementation

The Flask backend initializes the database, exposes REST endpoints, and serves the static frontend pages. Password hashes are stored instead of plain text passwords. Database records are inserted and updated using helper functions that open and close PostgreSQL connections safely.

- `init_db` creates tables automatically when the first request arrives.
- The users table includes reset code and expiry columns for password recovery.
- The applications table stores both the AI prediction and the admin review outcome.
- Notifications are created when review status changes.

14.3 Machine Learning Implementation

The ML training script uses a preprocessing pipeline and Logistic Regression model. The trained model is saved in ML/artifacts and loaded by the service class in Backend/ml_service.py. The runtime service checks whether the artifact exists before serving prediction requests.

14.4 AI Implementation

The AI service constructs prompts using the student profile and model output, then sends them to an external chat completion endpoint when configured. If the call fails or the key is missing, the service produces a stable fallback explanation based on the student profile.

15. Results and Screenshots

The current implementation demonstrates the end-to-end workflow of scholarship management: registration, login, application submission, prediction, review, and reporting. The admin dashboard shows summary statistics such as total applications, approved applications, rejected applications, and average eligibility probability.

The student dashboard displays submission history and the latest AI-generated result. The admin dashboard displays a review queue where each row contains the student name, prediction, probability, status, and review action buttons.

The following screenshots should be inserted in the final submission if available: landing page, login page, student dashboard, application form, admin dashboard, review queue, AI insights panel, and exported report preview.

- Login page: secure access for students and administrators.
- Student dashboard: application submission and personal history.
- Admin dashboard: application review and summary statistics.
- AI insights: generated explanations and report summaries.

16. Conclusion

The AI-Enhanced Scholarship Management System successfully modernizes scholarship processing by combining web technologies, machine learning, and generative AI. It reduces manual workload, improves data organization, and makes the approval process more transparent.

The project demonstrates how a compact full-stack application can still provide meaningful intelligence through predictive analytics and AI-generated narratives. By keeping the workflow simple and structured, the system remains easy to deploy and extend.

17. Future Enhancements

- OCR-based verification of uploaded certificates and income documents.
- More advanced scholarship recommendation engine across multiple schemes.
- SMS or email notifications for application status changes.
- Role-based audit logs and approval history tracking.
- Mobile app version for easier student access.
- Advanced explainability using SHAP or similar interpretation methods.
- Better integration with real institutional scholarship datasets.

18. References

Flask Documentation - <https://flask.palletsprojects.com/>

PostgreSQL Documentation - <https://www.postgresql.org/docs/>

scikit-learn Documentation - <https://scikit-learn.org/stable/>

Pandas Documentation - <https://pandas.pydata.org/docs/>

NumPy Documentation - <https://numpy.org/doc/>

Werkzeug Security Utilities - <https://werkzeug.palletsprojects.com/>

Python Requests Documentation - <https://requests.readthedocs.io/>

Project Repository - <https://github.com/maheshkomakula/AI-Enhanced-Scholarship-Management-System.git>

Appendix: Key Repository Components

The repository includes the following major folders and files: Frontend/, Backend/, ML/, api/, requirements.txt, Dockerfile, Procfile, vercel.json, and deployment instructions in DEPLOY.md. The ML folder contains the training notebook, dataset, trained model artifact, and metrics file. The Backend folder contains the Flask application, database helpers, AI service, email service, ML service, and API routes.